

# Project A4: Concurrent Data Structures and Memory Coherence

## 1 Objective

This project measures how lock granularity affects scalability in a shared concurrent hash table. A coarse-grained locking design is compared against a striped-lock design under multiple workloads, key set sizes, and thread counts.

## 2 Methodology

### 2.1 System

Experiments were run on an Intel Core i7-1260P system (16 hardware threads) on Linux 6.8. All tests use one shared hash table instance accessed by all threads.

### 2.2 Workloads and Parameters

Three workload mixes are evaluated:

- **Lookup-only:** read-dominated workload.
- **Insert-only:** writer stress test.
- **Mixed:** combined reads and writes (mixed workload in the current plots is 50/50).

Data set sizes are  $10^4$ ,  $10^5$ , and  $10^6$  keys. Thread counts are 1, 2, 4, 8, and 16 threads.

### 2.3 Implementations

- **Coarse:** one global mutex protects the entire table.
- **Striped:** multiple locks protect independent stripes (bucket groups).

## 3 Reproducibility

Commands:

```
make -j
scripts/run_sweep.sh
# or with perf:
scripts/run_sweep_perf.sh
python3 plots/plot_a4.py --in data/processed/results.csv --outdir data/processed
```

Each configuration was executed multiple times and averaged. The following metrics are reported:

- Throughput (ops/s)
- Speedup relative to 1 thread

## 4 Results and Discussion

### 4.1 Lookup-only (read-dominated)

Figures 1 and 2 show lookup-only behavior. Striped locking scales strongly up to 4–8 threads across all key sizes. Coarse locking slows down as threads increase due to global lock contention.

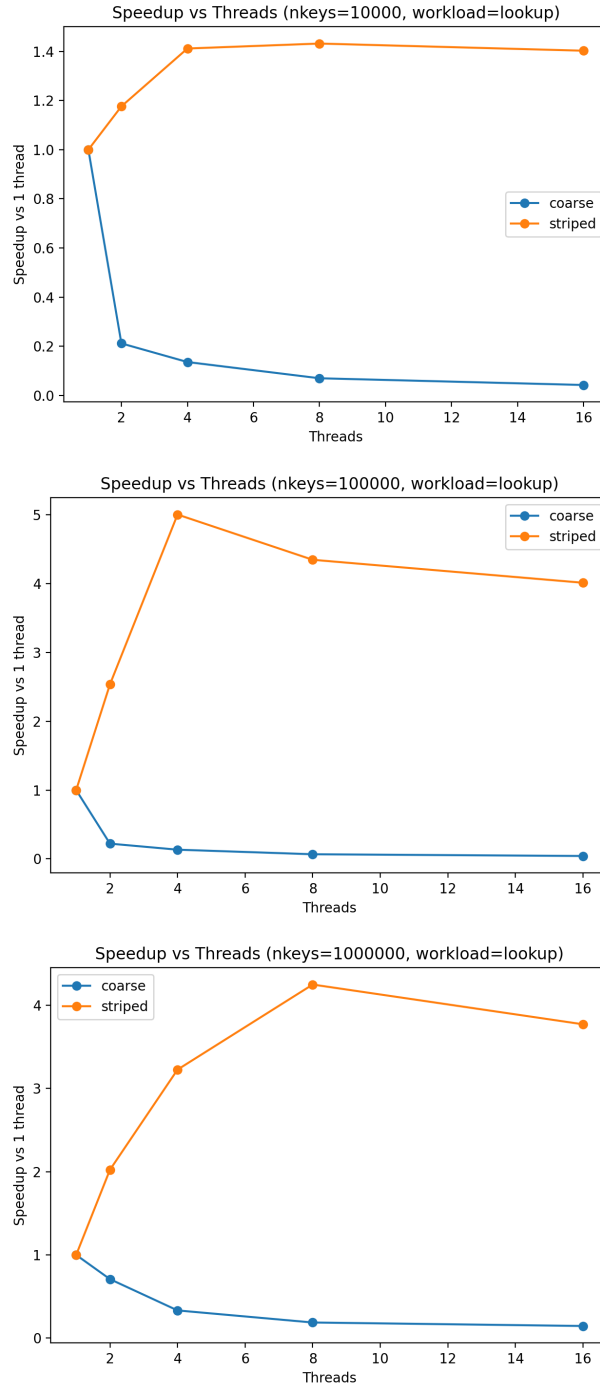


Figure 1: Speedup vs threads for lookup-only workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

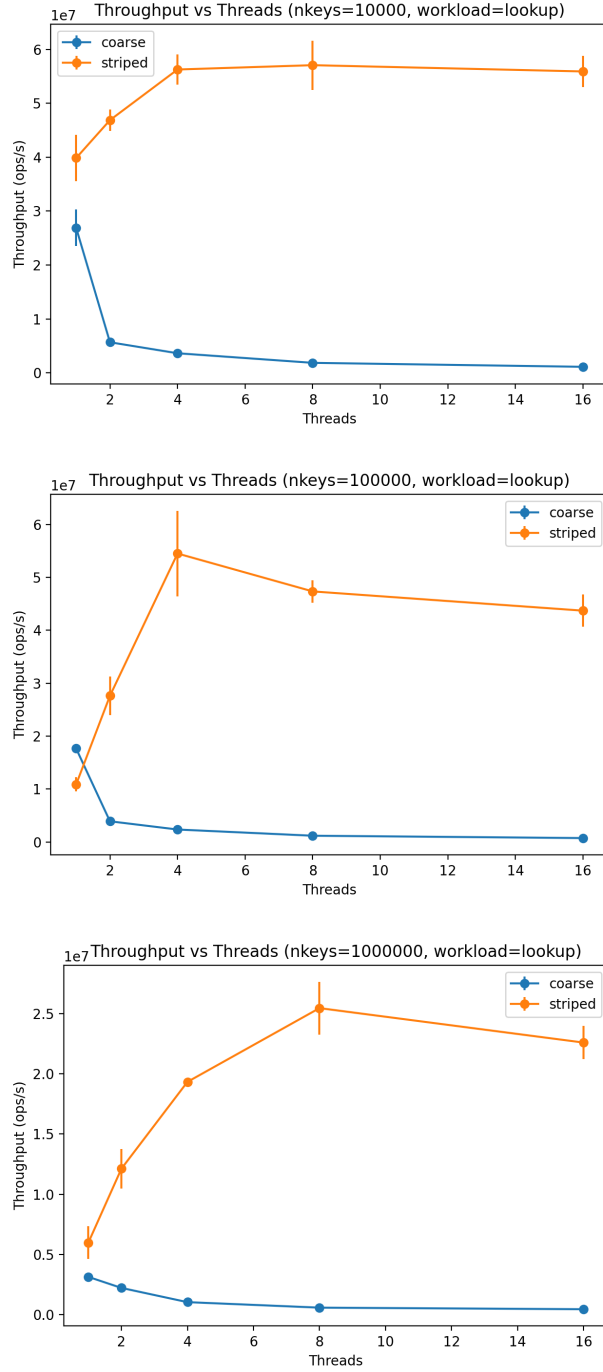


Figure 2: Throughput vs threads for lookup-only workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

Lookup operations are short and benefit from parallelism when contention is reduced. A single global lock serializes all threads, so coarse locking loses throughput and speedup as threads increase. Striping allows independent buckets to proceed in parallel, so scaling improves until shared-cache and coherence overheads limit gains.

## 4.2 Insert-only (writer stress test)

Figures 3 and 4 show insert-only behavior. Striped locking still scales, but less than lookup-only because inserts cause more coherence traffic and higher lock pressure. Coarse locking performs poorly because every insert is serialized.

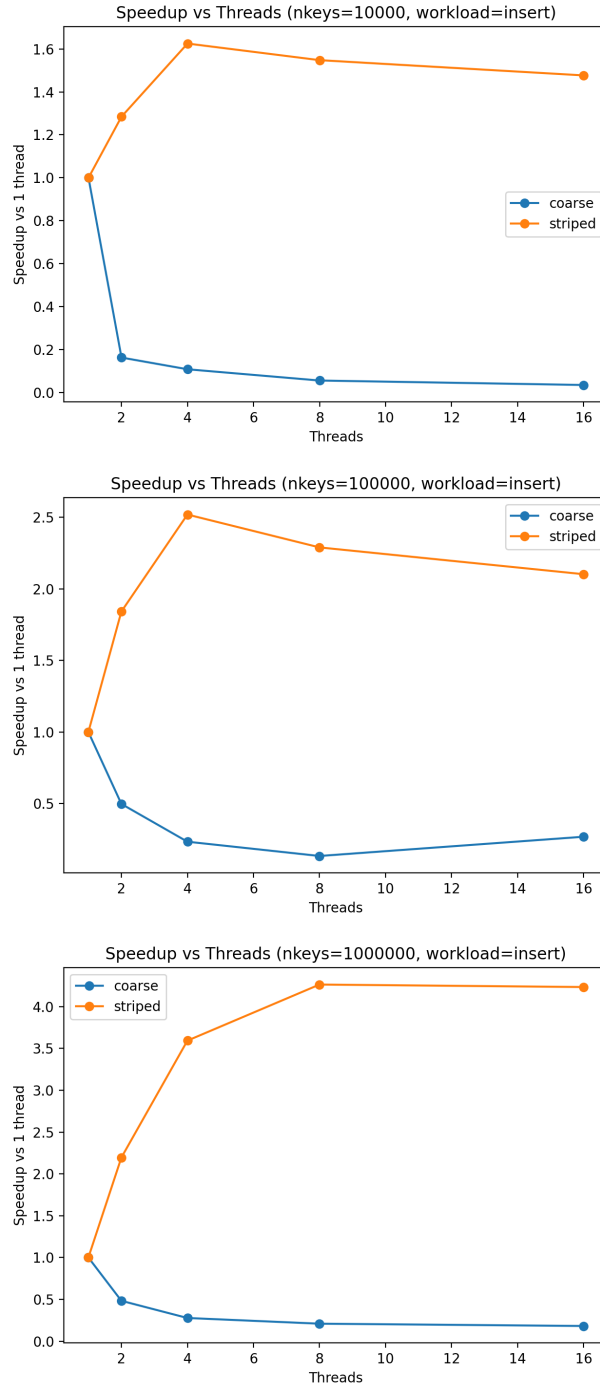


Figure 3: Speedup vs threads for insert-only workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

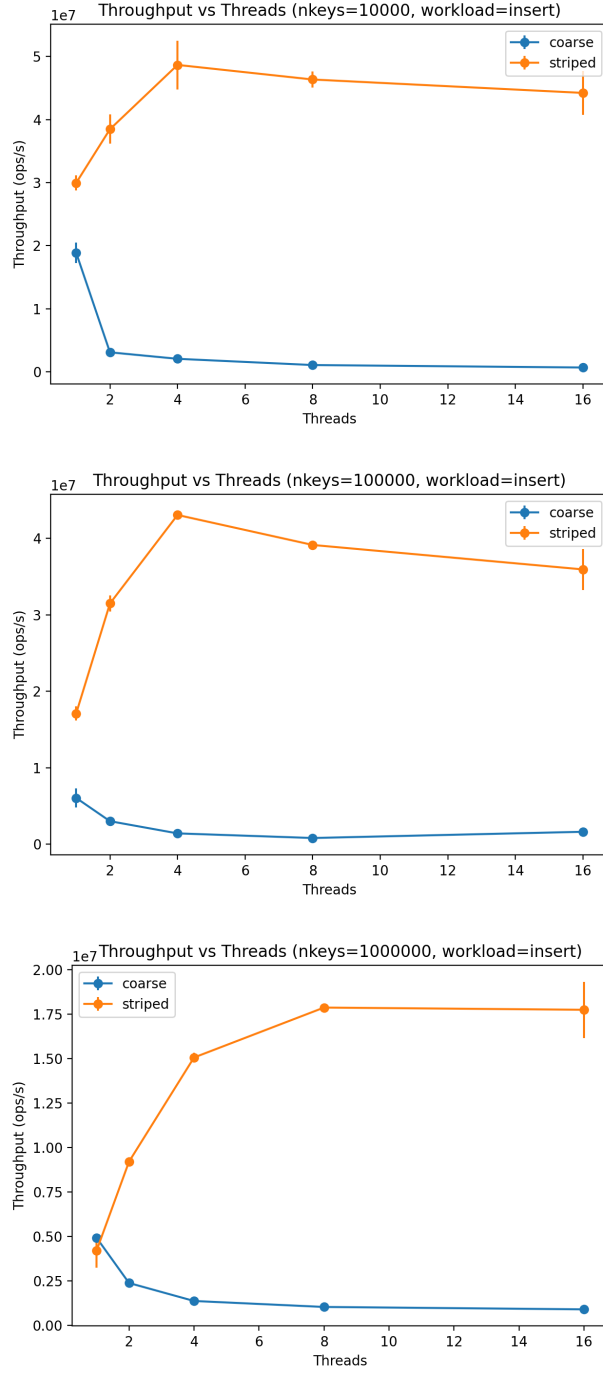


Figure 4: Throughput vs threads for insert-only workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

Inserts update shared state, which increases cache line invalidations and coherence work. Striped locking contains contention to the accessed stripe, but saturation still occurs at higher thread counts. Coarse locking forces single-thread progress, causing throughput collapse as threads increase.

### 4.3 Mixed workload (reads + writes)

Figures 5 and 6 show mixed behavior. Striped locking outperforms coarse locking across all sizes. Scaling is between lookup-only and insert-only because write operations introduce contention and coherence overhead.

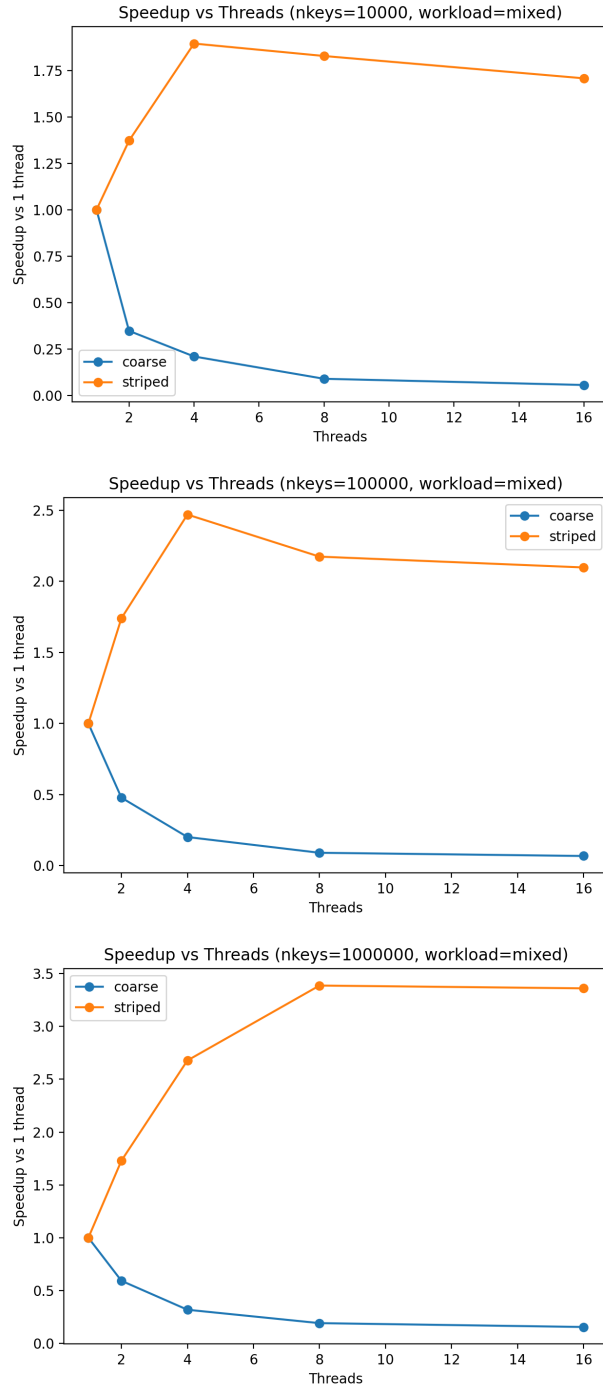


Figure 5: Speedup vs threads for mixed workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

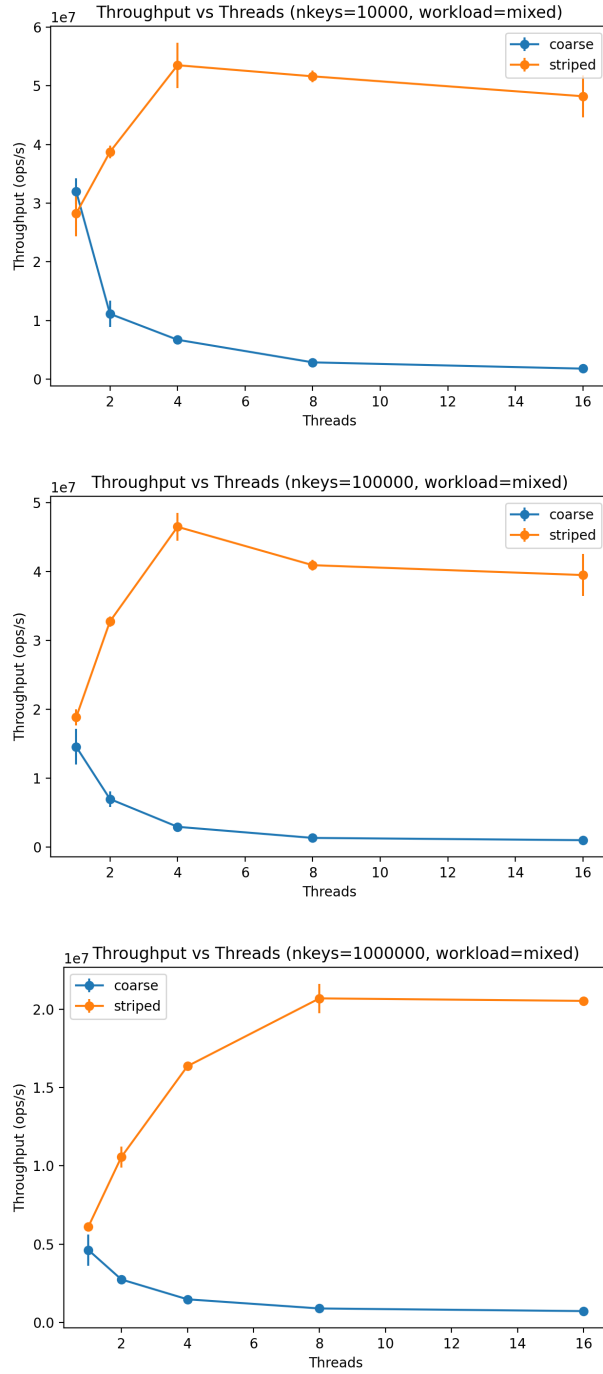


Figure 6: Throughput vs threads for mixed workload (top to bottom:  $10^4$ ,  $10^5$ ,  $10^6$  keys)

Mixed workloads preserve some read parallelism, but writes add contention and cache invalidations. Striping reduces the scope of blocking, so it remains scalable compared to a global lock.

#### 4.4 Impact of Data Set Size ( $10^4$ to $10^6$ )

Across all workloads, striped locking benefits more as key set size increases. Larger key sets spread accesses across more buckets, reducing the chance that threads contend on the same lock. Coarse locking does not benefit from this effect because the entire table is still protected by one lock.

#### 4.5 Thread Scaling Summary (1, 2, 4, 8, 16)

Coarse locking generally shows speedup below 1 at 2+ threads, indicating negative scaling. Striped locking improves through 4–8 threads and then saturates due to lock overhead and shared-cache limits.

#### 4.6 Perf Counter Analysis

Hardware performance counters were collected using `perf stat` during benchmark execution. Cycles were recorded for all configurations and normalized as cycles per operation.

**Cycles per Operation.** For the coarse-grained design, cycles per operation increase sharply as thread count rises. At 1 thread, the table achieves its lowest cycles/op baseline. At 16 threads, total throughput drops by more than 5×, while cycles/op increase substantially. This indicates that threads spend most of their time waiting on the global lock rather than performing useful work.

In contrast, the striped design maintains significantly lower cycles per operation at higher thread counts. Although cycles/op still increase due to cache coherence traffic and lock overhead, the increase is much smaller than in the coarse-grained case. This explains why striped locking continues to scale up to 4–8 threads before saturating.

**Cache and Coherence Effects.** While LLC load and store misses were collected where supported, the observed throughput and cycles/op trends are already consistent with increased coherence traffic. In the coarse-grained design, frequent invalidation of the cache line containing the global mutex causes lock bouncing and pipeline stalls. Striped locking distributes synchronization across multiple cache lines, reducing coherence hot spots and improving parallel efficiency.

Overall, perf counters confirm that lock contention and cache coherence dominate performance at higher thread counts, and explain the throughput collapse of coarse-grained locking.

### 5 Conclusion

Lock granularity is the main factor controlling scalability in this shared hash table. Coarse-grained locking serializes all operations and degrades as threads increase. Striped locking reduces contention by allowing parallel progress across independent stripes, improving throughput and speedup for lookup, insert, and mixed workloads across  $10^4$ – $10^6$  keys.